

Chapter 0

Java Basics

This chapter introduces the syntax of the Java programming language, alongside comparable constructs in the Python programming language for those familiar with the latter but not with the former. This book restricts itself to use only constructs that are available in Java 8, as that is a stable version widely used in the software industry, even if it is fairly ancient. I assume that you have installed some suitable Java Development Kit (JDK) and some programming-oriented text editor, such as BBEdit (for macOS) or Sublime Text (for any system). Note that Microsoft Word and Google Docs are word processors, *not* text editors, and absolutely should not be used for this purpose. Apple’s TextEdit is fine, if a bit barebones, as long as you set the document format to plain text rather than rich text with “Format > Make Plain Text”.

0.1 The Structure of a Program

In Java, a program consists of one or more *classes*. For now, you can think of a *class* as a logical grouping of related functions and variables. Functions that belong to a class are called *methods*, so I will use the terms more or less interchangeably. The basic shape of a program is something like this, then:

```
<SampleProgram.java 1>≡  
    public class SampleProgram {  
        <contents of the SampleProgram class 2a>  
    }
```

1

The “<SampleProgram.java 1>≡” line is not part of the program; it is part of the navigation in this book. That indicates that this piece of code is named “SampleProgram.java” and is defined here, on page 1. Similarly, the line consisting of “<contents of the SampleProgram class 2a>” is not part of the program, it means that you should insert there a piece of code described later, the first on page 2. The other two lines are actual code, and there’s already a lot going on here! This

says that we are defining a class called `SampleProgram` (the name of the class *must* exactly match the name of the file), and this class is `public`: it is visible to other classes, like the Java system itself, and therefore it is runnable. The curly braces “{...}” indicate the contents of the class; this is the kind of grouping that is indicated by indentation in Python.

When the Java system runs a program, it loads the specified class (for example, our `SampleProgram`) and looks for a method named “`main`” that belongs to the class, which must take exactly one parameter of type “`String[]`”, traditionally named “`args`”, and return nothing. To make our sample program write the text “Hello, world!” to the screen, we should define our `main` method as follows:

```
2a <contents of the SampleProgram class 2a>≡ (1)
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
```

Here again, “`public`” means that this method should be visible and the curly braces “{...}” indicate grouping. The “`static`” means that the method belongs to the class itself; we will discuss non-static methods later. The “`void`” means that the method does not return anything, as required. The method is named “`main`” and it takes one parameter: “`String[] args`” means that this parameter is named “`args`” and the values allowed to be passed as “`args`” have type “`String[]`”. Finally, “`System.out`” is an object provided by Java itself, representing your program’s output to the terminal, and “`println`” is a method that belongs to that object; it writes the provided value and ends the line. The semicolon (“;”) is not optional. It marks the end of the statement.

As mentioned, this piece of code should be placed inside the class defined on page 1. Putting it together results in the following file:

```
2b <SampleProgram.java 2b>≡
    public class SampleProgram {
        public static void main(String[] args) {
            System.out.println("Hello, world!");
        }
    }
```

The first two and last two lines are *boilerplate*: a template of code that forms part of every program (with the class name changed as appropriate). The only thing unique to this program is the single line that does the writing. In Python, an equivalent program would be simply:

```
2c <SampleProgram.py 2c>≡
    if __name__ == '__main__':
        print("Hello, world!")
```

In order to *execute* the Java program, we must compile it. This translates the Java source code into another form, the native code of the JVM (Java Virtual

Machine). After creating the file, open a terminal in its containing folder and run the following command:

```
javac SampleProgram.java
```

After this runs, there should be a new file called “SampleProgram.class”. That is the compiled version of the class. If the program contains other classes, they too should be compiled; to compile every Java file in the folder at once, use this instead:

```
javac *.java
```

The `*.java` is a wildcard expanded by the system into the names of every file whose name ends in “.java”. In order to actually run the program, use the following command:

```
java SampleProgram
```

This tells Java to load the “SampleProgram.class”, find its “main” method, and run that. If all went well, you should see “Hello, world!” as output.

0.2 Identifiers

We have seen a class called “SampleProgram”, a method called “main”, and a parameter called “args”. There are rules for these *identifiers*: they must begin with a letter, and may contain letters, digits, and the underscore character. Because the name of a class must be a valid identifier, so too must be the base name of the file that contains it (when you ignore the “.java” extension). You can have a program named “four_by_four.java”, but not one named “4x4.java”. Additionally, there are some words that would be valid but are *reserved* by Java and therefore unusable. These reserved words are as follows:

- | | | | | |
|------------|-----------|--------------|----------------|-------------|
| • abstract | • default | • goto | • package | • this |
| • assert | • do | • if | • private | • throw |
| • boolean | • double | • implements | • protected | • throws |
| • break | • else | • import | • public | • transient |
| • byte | • enum | • instanceof | • return | • true |
| • case | • extends | • int | • short | • try |
| • catch | • false | • interface | • static | • void |
| • char | • final | • long | • strictfp | • volatile |
| • class | • finally | • native | • super | • while |
| • const | • float | • new | • switch | |
| • continue | • for | • null | • synchronized | |

0.3 Types and Variables

A *variable* is, abstractly, a named location for storing values. Every value has a *type*, which determines how it is represented in the computer's memory. In Python, variables are unrestricted, so an expression like “`x - 2`” may or may not be erroneous, depending on what value “`x`” refers to when this expression is encountered. If it is an integer value like `10`, then the expression evaluates to the number 8, but if it is a string value like `"hello"`, then evaluating the expression results in an error.

Java avoids this problem by annotating the creation of variables with a *type* restriction. In Python, you assign a variable like “`x = 2`”, and it is created if it does not already exist. Java separates these aspects. You create a variable by stating the type of values it is allowed to hold, then its name, and then optionally giving it an initial value before ending the statement with a semicolon. We'll discuss types more in a bit, but for now, let's just use “`int`”, one of the types that represents integer values. The following code snippet creates two variables, “`x`” and “`y`”, and assigns to them the values 6 and 7, respectively.

```
4 <variable tests 4>≡
    int x;
    int y = 7;
    x = 6;
```

For “`y`”, the creation and assignment are combined. For “`x`”, they are separated. Before the “`x = 6;`” line, “`x`” has no value, and using it in an expression is erroneous. The compiler will detect if you are trying to use a variable in a context that is nonsensical given its type, or when it has no value, and it will tell you that there is a problem before even letting you try to run the incorrect code. It will not compile. Initially, you might find this frustrating, but catching these kinds of errors at compile-time instead of at runtime can save hours of headache!

The first two lines of the above code snippet are *declarations*; they *declare* that the variables exist and state their type restrictions. The first assignment to a variable is its *initialization*, so the last two lines are initializations. Subsequent assignments do not have special names.

When you create a variable, a little piece of memory is obtained for that name to be bound to. Java has two kinds of values: *primitive types* store the value directly in that little piece of memory, while *reference types* store the location of another piece of memory, which itself contains the value. You can think of the latter kind as if the variable refers to the tail-end of an arrow whose head-end points to the data. (In Python, everything is of reference type.) The following are available as primitive types in Java:

- `boolean`: either `true` or `false`
- `char`: a UTF-16 code unit (usually: a Unicode character)

- **byte**: an 8-bit signed integer, from -128 through $+127$
- **short**: a 16-bit signed integer, from $-32,768$ through $+32,767$
- **int**: a 32-bit signed integer, from $-2,147,483,648$ through $+2,147,483,647$
- **long**: a 64-bit signed integer, from $-9,223,372,036,854,775,808$ through $+9,223,372,036,854,775,807$
- **float**: a 32-bit single-precision floating-point number conforming to the IEEE 754 standard
- **double**: a 64-bit double-precision floating-point number conforming to the IEEE 754 standard

By default, numbers written without a decimal point are of type `int`, and those with a decimal point are of type `double`. To get a `long`, suffix your integer number with a capital `L` or lowercase `l`, like `1903L` is a `long` with value $+1903$. To get a `float`, suffix your number with a capital `F` or lowercase `f`, like `3.14f` is a `float` with value $+3.14$.

In some contexts, Java requires a reference type. There are reference-type “wrappers” for each of these, whose names are capitalized and spelled in full: `Boolean`, `Character`, `Byte`, `Short`, `Integer`, `Long`, `Float`, and `Double`. Converting a primitive type to a wrapped type is called *boxing* and the reverse is called *unboxing*. In many contexts, this is done automatically. However, you can box and unbox values explicitly as follows.

<manual boxing 5a>≡

```
Integer i = Integer.valueOf(42);  
int j = i.intValue();
```

5a

The `valueOf` method of the `Integer` class takes an `int` and returns a reference to a boxed form. Its `intValue` method takes no arguments and returns a copy of the unboxed value. (The first is a static method, belonging to the class itself, while the second is a non-static method, belonging to the object `i`. We will further explore this distinction later.)

For primitive types, assignment copies the value. For reference types, assignment copies the *arrow*, so multiple names can refer to the same object. A *box-and-arrow diagram* represents variables and their values this way. For instance, the following code snippet initializes four variables, `i`, `j`, `x`, and `y`, and the result is represented by the diagram in Figure 1.

<initializing variables 5b>≡

```
Integer i = Integer.valueOf(42);  
Integer j = i;  
int x = 12;  
int y = x;
```

5b

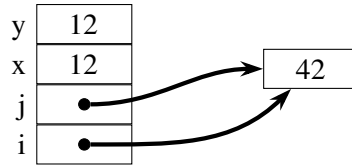


Figure 1: A simple box-and-arrow diagram.

These diagrams can be augmented with type information if there is ambiguity. They can represent connections within and among complicated classes. It is important to understand how to read them and how to construct the systems they represent.

0.4 Some Expressions

A *literal* is a specific value typed directly into the code, like numbers 42 or 1903L, or like Boolean values `true` or `false`. Integer literals can contain underscores for readability. A `char` is given in single-quotes, like `'A'`, while a `String` is given in double-quotes, like `"Hello!"`. Special characters can be obtained using *escape sequences* that begin with the backslash character; as this steals that character away, it requires two backslashes to encode a single backslash! Some of the special characters defined that way are as follows:

Sequence	Meaning
<code>\t</code>	tab
<code>\b</code>	backspace
<code>\n</code>	newline
<code>\r</code>	carriage return
<code>\f</code>	form feed
<code>\'</code>	single-quote
<code>\"</code>	double-quote
<code>\\</code>	backslash

If you happen to know the Unicode value of a character, you can enter it directly with an escape sequence as well. For instance, the Greek letter omega (ω) has Unicode value 03C9, so you can enter it as `'\u03C9'`.

Literals and variables both represent values of certain types. There are several operators available that combine values of appropriate types. For instance, arithmetic operations on integers include addition (+), subtraction (-), multiplication (*), division (/), and remainder (%). The last of these is also known as *modulo*. There is no exponentiation operator. Python provides two division operators: a single slash for true division and a double slash for integer division; in contrast,

Java has only one operator, and chooses which division to use based on the types of the operands. That is, $5/2$ is 2 while $5.0/2.0$ is 2.5 .

Values may be converted to a compatible type by *casting*. To do so, place the target type in parentheses before a value. So `(double)5/2` is 2.5 . Note that the cast has higher precedence than the division. If instead we said `(double)(5/2)`, the result would be 2.0 . In some situations, Java will do casting implicitly. For instance, in $5.0/2$, the integer 2 is implicitly promoted to a double-precision floating-point number like 2.0 so that the types match, and then the operation is performed, giving the output 2.5 .

The addition operator works as *concatenation* on strings: `"dead" + "bolt"` is `"deadbolt"`. (This is not an efficient operation, as data must be copied from both strings into the new larger output.) The `+` operator can actually take any primitive type alongside a `String`, in which case the value of primitive type is first converted to a `String` to match. This is a special case! Values of primitive type are not normally compatible with `String` and cannot be converted by a cast like `(String)3`. Instead, you need to use a static method: `String.valueOf(3)`. Every object of reference type has a `toString` method that does this conversion: if `x` is of reference type, then `x.toString()` is a representation of `x` as a `String`. Multiplication cannot take a `String` argument.

0.5 Defining Functions

Like the main method of a program, every function must state what type of values it expects and returns. Suppose we want to write a function that takes a number and returns its square. To return a value, we use the `return` keyword. We need to choose what *kind* of number we want to take. For instance, we might want to square `int` values:

```
<int square 7a>≡
    public static int square(int x) {
        return x*x;
    }
```

(8a) 7a

Or we might want to square double values:

```
<double square 7b>≡
    public static double square(double x) {
        return x*x;
    }
```

(8a) 7b

In fact, we can define both of these functions in the same class, and the type of the parameters determines which is called. This is one way of being able to specify *polymorphism*: accepting multiple types of values and potentially specifying different behavior based on the type of data provided. The following sample program demonstrates this. Compile and running it, and you should notice that

the first print-statement emits 25, an integer, while the second emits 25.0, a floating-point value.

```
8a <SquareTest.java 8a>≡
    public class SquareTest {
        <int square 7a>
        <double square 7b>
        public static void main(String[] args) {
            System.out.println(square(5));
            System.out.println(square(5.0));
        }
    }
```

There is, however, an issue with one of these implementations of the squaring function. What would happen if instead of 5, we used 50_000? What can we do about it?

0.6 Control Flow

By default, program execution flows directly from top to bottom, starting in the main method. Calling other methods interferes with this order, pausing to execute the code in the method called before resuming with its return value, if any, accounted for. There are a few other ways to modify the default execution order.

0.6.1 Conditionals: If-Statements

A *condition* is any boolean value. These can be combined with the operators && (meaning “and”), || (meaning “or”), and ! (meaning “not”). An if-statement is written as follows:

```
8b <if-then 8b>≡
    if (condition) {
        body
    }
8c>
```

Compare this to the Python equivalent:

```
8c <if-then 8b>+≡
    if condition:
        body
<8b 9a>
```

In each example, *condition* refers to a boolean expression and *body* is a sequence of one or more statements. The body is executed if, and only if, the condition evaluates to true.

Optionally, there can be an *else* block that says what to do if the condition evaluated to false. On either side, single-statement bodies need no braces, so

`else { if (...) { ... } }` collapses to just `else if (...) { ... }`. A fuller example might be something like the following:

```
<if-then 8b>+≡                                     <8c 9b> 9a
    if (condition1) {
        body1
    } else if (condition2) {
        body2
    } else if (condition3) {
        body3
    } else {
        body4
    }
```

Compare this to the Python equivalent:

```
<if-then 8b>+≡                                     <9a 9b
    if condition1:
        body1
    elif condition2:
        body2
    elif condition 3:
        body3
    else:
        body4
```

Comparison of primitive types is done using the operators `<`, `<=`, `==`, `>=`, `>`, and `!=`, which mean, in order, less than, less than or equal to, equal to, greater than or equal to, greater than, and not equal to. These comparisons are not appropriate for reference types. We will discuss the correct comparisons for those later. An integer is even if its remainder after division by two is zero. So we could define a function like the following:

```
<printEvenness 9c>≡                                     9c
    public static void printEvenness(int n) {
        if (n%2 == 0) {
            System.out.println(n + " is even!");
        } else {
            System.out.println(n + " is odd!");
        }
    }
```

Think about how you would test whether `n` is a multiple of either 3 or 5, with or without using connectives like `||`. What about testing whether `n` is a multiple of both 3 and 5, with or without using connectives like `&&`? You should be able to find multiple ways to do each of these.

0.6.2 Looping Constructions

Method calls and if-statements are all that you really need, but looping constructs are convenient. A while-loop is like an if-statement but the body executes zero or more times until the condition is false. If you forget to update the relevant variables, then you will be caught in an infinite loop!

```
10a <while-loop 10a>≡  
    while (condition) {  
        body  
    }
```

A for-loop has three components: an initializer, a condition, and an updater. The following two code snippets are operationally identical:

```
10b <for-loop 10b>≡  
    for (initializer; condition; update) {  
        body  
    }
```

```
10c <for-loop as while-loop 10c>≡  
    {  
        initializer;  
        while (condition) {  
            update;  
        }  
    }
```

There is also a variant of for-loops, called “enhanced for loops”, that iterate over certain kinds of container objects. We will not discuss enhanced for-loops for quite some time.

0.7 Comments and Documentation

Java has line comments, which begin at `//` and continue to the end of the line. It also has block comments, which begin at `/*` and continue until `*/`. Generally speaking, it is good practice to use line comments for quick explanations of unintuitive code snippets, while block comments are good for explaining the purpose of a complex piece of code.

Java comes with a tool, called Javadoc, specifically designed for documenting your system’s public API. With an extra star, opening as `/**` instead of just `/*`, you signal to this tool that a comment is documentation meant for *users* of your software, rather than for *developers* working in your code. Unlike Python docstrings, Javadoc documentation comments are *not* freeform text meant to be read in a text editor. Instead, they are HTML-based formatted descriptions, with a specific format, that the javadoc tool transforms into documentation viewable in a web browser. Build

your documentation with the following command, then look for “index.html” inside the generated “docs” folder:

```
javadoc -d docs *.java
```

Errors here indicate that your documentation is fatally flawed and cannot be built. Warnings usually indicate that you missed documenting something that you should have documented, although by default there are some warnings that do not indicate issues for program classes. The following sample code demonstrates some of the possibilities.

(JavadocDemo.java 11)≡

11

```
/** Demonstrate Javadoc documentation.
 * This class is just a sample.
 * It demonstrates Javadoc documentation comments.
 * @author Dakotah Lambert
 */
public class JavadocDemo {
    /** Write a simple welcome message.
     * @param args  command-line arguments (ignored).
     */
    public static void main(String[] args) {
        System.out.println("Welcome to Javadoc!");
    }
    /** Find the area of a square with the given side length.
     * @param n  the side length.
     * @return the product of n by itself.
     */
    public static int square(int n) {
        return n*n;
    }
}
```